

Exact Discrete Fixed-Point Adjoint for Thermal SIMPLE Topology Optimisation

Mathematical derivation, implementation mapping, and validation identities

Fira Software Ltd

19 July 2026

Revision 1.1: clarified thermal-property, coupling, and non-orthogonal scope

Status and precise scope. This note derives the production-qualified flow fixed-point sensitivity

$$\frac{dJ}{d\beta} = J_\beta + \psi^T M_\beta, \quad (I - M_x)^T \psi = J_x,$$

for the guarded OpenFOAM v2512 branch: serial execution, non-coupled patches, the laminar **Stokes** model, consistent SIMPLE, one pressure correction per map (no non-orthogonal pressure-corrector loop), the supported bounded Gauss-upwind thermal convection branch, and a reduced thermal objective. The coupling is one-way from flow to temperature: there is no buoyancy, no temperature-dependent viscosity or momentum coefficient, and no other return path from T to the SIMPLE flow map. The thermal transpose is exact for the assembled constant-property, orthogonal/uncorrected thermal operator used by the validated branch. With temperature-dependent properties it is the exact transpose of the frozen-property Picard operator, not the full nonlinear thermal Jacobian. With a nonzero explicit non-orthogonal diffusion correction, an additional correction-flux reverse is required. At lower-bound relaxation ties, the reported design sensitivity is the feasible right derivative, not a general two-sided derivative.

Abstract

The implementation differentiates the actual discrete SIMPLE iteration rather than only the continuous PDE or the assembled steady residual. The full flow state is $x = (U, p, \phi_{\text{int}}, \phi_{\text{bnd}})$. An exact reverse of one SIMPLE map provides M_x^T ; a matrix-free GMRES solve then obtains the fixed-point adjoint, following the reverse-accumulation principle for converged fixed points [1]. The design derivative $M_\beta^T \psi$ is obtained by reversing the beta-dependent implicit Brinkman coefficient through OpenFOAM matrix relaxation, the predictor, consistent-SIMPLE pressure correction, and the topology interpolation chain. A separate face-based derivation supplies the direct conductivity derivative J_β for the validated orthogonal/uncorrected thermal diffusion branch. The production implementation replaces the stock Brinkman sensitivity with $M_\beta^T \psi$, retains the direct conductivity term, and avoids double counting. References for fixed-point reverse accumulation, OpenFOAM adjoint methods, and Brinkman fluid topology optimisation are listed at the end of this note.

Contents

1	Primal model, design variables, and notation	4
1.1	Continuous equations represented by the finite-volume solver	4
1.2	Algebraic finite-volume state and pairing	4
2	Exact discrete thermal adjoint and flux objective seed	4
2.1	Discrete thermal residual, nonlinear Jacobian, and implemented transpose	4
2.2	Derivative of bounded Gauss-upwind convection with respect to face flux	5
2.3	Reduced thermal objective seed for the flow fixed point	5

3	The exact one-step consistent-SIMPLE map	6
3.1	Momentum predictor	6
3.2	adjustPhi map	6
3.3	Consistent-SIMPLE coefficient and pressure correction	6
4	Reverse of the pressure projection and consistent-SIMPLE algebra	7
4.1	Projection transpose at frozen coefficients	7
4.2	Pressure-coefficient reverse	7
4.3	Consistent coefficient algebra	8
5	Reverse of the momentum predictor	8
5.1	Predictor residual identity	8
5.2	Residual reverse	8
5.3	Diagonal and H_1 dependence on momentum flux	9
5.4	Flux interpolation transpose	9
5.5	Reverse of adjustPhi	9
6	One-step transpose and fixed-point adjoint	10
6.1	Exact one-step reverse	10
6.2	Fixed-point tangent equation	10
6.3	Fixed-point adjoint equation	10
6.4	Solved-adjoint identity	11
7	Analytic design reverse $M_\beta^T \psi$	11
7.1	Where beta enters the one-step map	11
7.2	Feasible derivative at structural lower-bound ties	11
7.3	Reverse trace and coefficient contraction	12
7.4	Mapping to physical beta and optimiser variables	12
8	Direct conductivity derivative J_β	12
8.1	Internal-face discrete derivative	12
8.2	Boundary-face derivative	13
8.3	Chain through the conductivity interpolation	13
9	Production assembly and avoidance of double counting	13
10	Validation identities and numerical evidence	14
10.1	Tangent residual-correction identity	14
10.2	All-cell design-direction identity	14
10.3	Bound-cell formulas	14
10.4	Production qualification summary	15
11	Implementation map	15

12 What is exact, and what remains branch-dependent	16
13 Final formula	16
References	17
A Compact derivation of the central fixed-point result	17

1 Primal model, design variables, and notation

1.1 Continuous equations represented by the finite-volume solver

The steady incompressible flow and thermal equations can be written schematically as

$$\nabla \cdot \mathbf{u} = 0, \quad (1)$$

$$\nabla \cdot (\mathbf{u} \otimes \mathbf{u}) - \nabla \cdot \boldsymbol{\tau}(\mathbf{u}) + \nabla p + a(\beta)\mathbf{u} = \mathbf{0}, \quad (2)$$

$$C(T) \nabla \cdot (\mathbf{u}T) - \nabla \cdot (D_{\text{eff}}(T, \beta) \nabla T) = 0. \quad (3)$$

Here $a(\beta)$ is the topology-dependent Brinkman coefficient, in the spirit of the porous-medium formulation used in fluid topology optimisation [3]. The effective thermal diffusivity represented by the implementation is

$$D_{\text{eff}} = D_f(T) + (D_s(T) - D_f(T))I_k(\beta) + (1 - I_k(\beta))\frac{\nu_t}{\text{Pr}_t}. \quad (4)$$

For the validated **Stokes** branch, $\nu_t = 0$.

The fixed-point state derived below is deliberately a *flow-only* state. The validated dependency graph is

$$\beta \longrightarrow (U, p, \phi) \longrightarrow T \longrightarrow J, \quad (5)$$

with no return arrow from T to the flow map. Thus the branch excludes buoyancy, $\nu(T)$, $\rho(T)$ in the momentum equation, temperature-dependent momentum sources, and any other thermal feedback into SIMPLE. If such feedback is introduced, T must be added to the fixed-point state and the corresponding cross-Jacobian blocks must be reversed.

The optimiser may use an underlying design variable ρ , a filter $\mathcal{F}$, a projection $\mathcal{P}$, and a physical field β :

$$\tilde{\rho} = \mathcal{F}\rho, \quad \beta = \mathcal{P}(\tilde{\rho}), \quad a = a(\beta), \quad I_k = I_k(\beta). \quad (6)$$

The OpenFOAM topology source's sensitivity post-processing applies the transpose of this chain. The derivations below first produce derivatives with respect to the local physical coefficient or physical β , after which the existing topology machinery maps them to the optimiser variable.

1.2 Algebraic finite-volume state and pairing

The exact SIMPLE map uses the full state

$$\mathbf{x} = (\mathbf{U}, \mathbf{p}, \phi_i, \phi_b), \quad (7)$$

where ϕ_i and ϕ_b are internal- and boundary-face fluxes. The algebraic state/seed pairing is

$$\begin{aligned} \langle \bar{\mathbf{x}}, \delta \mathbf{x} \rangle &= \sum_c \bar{U}_c \cdot \delta U_c + \sum_c \bar{p}_c \delta p_c \\ &+ \sum_{f \in \text{int}} \bar{\phi}_f \delta \phi_f + \sum_{f \in \text{bnd}} \bar{\phi}_f \delta \phi_f. \end{aligned} \quad (8)$$

No cell-volume factor is inserted into this pairing: the finite-volume matrices already use integrated coefficients. Volume and time-step normalisation are applied only when the final field is converted into OpenFOAM's **betaMult** convention.

2 Exact discrete thermal adjoint and flux objective seed

2.1 Discrete thermal residual, nonlinear Jacobian, and implemented transpose

In the most general temperature-dependent-property case, the converged thermal residual is

$$\mathcal{R}_T(T, \phi, \beta) = A_T(T, \phi, \beta)T - b_T(T, \phi, \beta) = 0. \quad (9)$$

For an objective $J(T)$, define

$$\mathcal{L}_T = J(T) + T_a^T \mathcal{R}_T(T, \phi, \beta). \quad (10)$$

The exact nonlinear thermal Jacobian is

$$K_T = \frac{\partial \mathcal{R}_T}{\partial T} = A_T + \left[\frac{\partial A_T}{\partial T} \right] T - \frac{\partial b_T}{\partial T}, \quad (11)$$

and its exact discrete adjoint is

$$\boxed{K_T^T T_a = -J_T}. \quad (12)$$

The implemented exact-scalar branch constructs and transposes the matrix assembled at the current property state. Denoting that frozen matrix by $A_T(\phi, \beta)$,

$$\mathcal{R}_T^{\text{Picard}} = A_T(\phi, \beta)T - b_T = 0, \quad \boxed{A_T^T T_a = -J_T}. \quad (13)$$

The implementation retains diagonal coefficients, swaps upper and lower coefficients, and handles known boundary/source terms consistently. Therefore eq. (13) is the exact discrete transpose of the assembled constant-property or frozen-property Picard operator. It equals the full nonlinear adjoint eq. (12) only when the omitted property-derivative terms vanish. The production-qualified claim in this note is restricted to that validated constant-property branch; a variable-property run must be described as a frozen-property/Picard derivative unless the terms in eq. (11) are added.

2.2 Derivative of bounded Gauss-upwind convection with respect to face flux

Consider an internal face f with owner P , neighbour N , and oriented flux ϕ_f . Let

$$A_P = C_P T_{a,P}, \quad A_N = C_N T_{a,N}, \quad (14)$$

and let T_f be the upwind temperature selected by the sign of ϕ_f . The bounded convection contribution to the thermal Lagrangian at that face can be written

$$\ell_f = \phi_f T_f (A_P - A_N) - \chi \phi_f (A_P T_P - A_N T_N), \quad (15)$$

where $\chi = 1$ for the bounded form and $\chi = 0$ for the unbounded form. Away from $\phi_f = 0$, the upwind branch is fixed, so T_f is locally constant with respect to ϕ_f . Hence

$$\boxed{g_{\phi,f} = \frac{\partial \mathcal{L}_T}{\partial \phi_f} = T_f (A_P - A_N) - \chi (A_P T_P - A_N T_N)}. \quad (16)$$

For a non-empty boundary face with adjacent cell P and boundary value T_b , the corresponding result is

$$\boxed{g_{\phi,f} = C_P T_{a,P} (T_b - \chi T_P)}. \quad (17)$$

At a zero-flux switching point, the upwind map is non-differentiable and no unique classical derivative exists. The implementation therefore validates and uses fixed branches away from such crossings.

2.3 Reduced thermal objective seed for the flow fixed point

After solving eq. (13), the reduced thermal objective depends on the flow state through the convecting flux. Therefore

$$J_x = (\mathbf{0}, 0, s_c g_{\phi,i}, s_c g_{\phi,b}), \quad (18)$$

where s_c is the implementation's coupling-sign convention. The implementation constructs this seed with `fixedPointObjectiveSeed()` from `thermalFluxSensitivity()`.

3 The exact one-step consistent-SIMPLE map

3.1 Momentum predictor

After momentum assembly, equation relaxation, and `fvOptions.constrain`, let

$$D_U = \text{UEqn.D}(), \quad r_{AU} = \frac{V}{D_U} = \frac{1}{\text{UEqn.A}()}, \quad Y = r_{AU} \text{UEqn.H}. \quad (19)$$

A crucial OpenFOAM identity, verified numerically and then used analytically, is

$$\boxed{Y = U + \frac{r_{AU}}{V} E, \quad E = \text{UEqn.residual}().} \quad (20)$$

The use of $D_U = \text{UEqn.D}()$, rather than only `diag()`, is essential because `D()` includes the component-averaged boundary-diagonal contribution used by `A()`.

Let $H_0 = \mathcal{C}_H(Y, U, p)$ denote `constrainHbyA`. The predictor face flux is

$$F_{0,f} = \mathbf{S}_f \cdot (w_f H_{0,P} + (1 - w_f) H_{0,N}) \equiv (\mathcal{B}H_0)_f. \quad (21)$$

Boundary faces use their actual patch value rule and value-internal coefficients.

3.2 adjustPhi map

Denote the result of `adjustPhi` by

$$F_1 = \mathcal{A}_\phi(F_0). \quad (22)$$

When pressure needs a reference and adjustable positive outflow exists, define the face classes

$$\begin{aligned} \mathcal{I} &= \{f : F_{0,f} < 0\}, \\ \mathcal{F}_+ &= \{f : F_{0,f} > 0 \text{ on a fixed-outflow patch}\}, \\ \mathcal{A}_+ &= \{f : F_{0,f} > 0 \text{ on an adjustable patch}\}. \end{aligned} \quad (23)$$

An adjustable face carrying negative flux belongs to $\mathcal{I}$, because OpenFOAM classifies the sign before patch adjustability. Then

$$I = - \sum_{f \in \mathcal{I}} F_{0,f}, \quad O_f = \sum_{f \in \mathcal{F}_+} F_{0,f}, \quad O_a = \sum_{f \in \mathcal{A}_+} F_{0,f}, \quad m = \frac{I - O_f}{O_a}. \quad (24)$$

Faces in $\mathcal{A}_+$ are multiplied by m ; all other faces remain unchanged. A face exactly at zero flux is a switching point and has no general classical derivative. In the common branch where `p.needReference() = false`, $\mathcal{A}_\phi$ is simply the identity.

3.3 Consistent-SIMPLE coefficient and pressure correction

For consistent SIMPLE,

$$\boxed{r_{AtU} = \frac{1}{1/r_{AU} - H_1}, \quad q = r_{AtU} - r_{AU}.} \quad (25)$$

The corrected predictor velocity and pressure-equation flux are

$$H = H_0 + q \mathcal{G}p, \quad (26)$$

$$F_2 = F_1 + \mathcal{I}(q) Np, \quad (27)$$

where $\mathcal{G}$ is the cell-gradient operator, $Np = \text{snGrad}(p)|S_f|$, and $\mathcal{I}$ is cell-to-face interpolation.

Let $C(r_{AtU})$ be the pressure-flux operator and D_f the face-divergence operator. The pressure solve can be represented as

$$L(r_{AtU})\tilde{p} = D_f F_2, \quad L = D_f C. \quad (28)$$

The new flux uses the unrelaxed pressure solution,

$$\phi^+ = F_2 - C\tilde{p}, \quad (29)$$

while the stored pressure is relaxed,

$$p^+ = \omega_p \tilde{p} + (1 - \omega_p)p. \quad (30)$$

The final velocity correction is

$$U^+ = H - r_{AtU} \mathcal{G} p^+. \quad (31)$$

Equations eqs. (19), (21), (24), (25) and (28) to (31) define the one-step map

$$\boxed{x^+ = M(x, \beta)}. \quad (32)$$

The map above contains one pressure solve and its final flux update. If the primal algorithm executes a non-orthogonal pressure-corrector loop, every corrector is part of M and must be included in both the forward map and its reverse. The production-qualified branch therefore requires zero non-orthogonal pressure correctors.

4 Reverse of the pressure projection and consistent-SIMPLE algebra

4.1 Projection transpose at frozen coefficients

Given a seed g on ϕ^+ , introduce

$$\mathcal{P} = g^T (F - Cp) + \lambda^T (Lp - D_f F). \quad (33)$$

Stationarity with respect to p gives

$$\boxed{L^T \lambda = C^T g}. \quad (34)$$

The seed on the incoming flux is

$$\boxed{\bar{F} = g - D_f^T \lambda}. \quad (35)$$

This is the exact discrete projection transpose used in the implementation.

4.2 Pressure-coefficient reverse

If the pressure-flux operator depends on a coefficient γ , then differentiating eq. (33) yields

$$\frac{\partial \mathcal{P}}{\partial \gamma} = -g^T C_\gamma p + \lambda^T L_\gamma p. \quad (36)$$

Since $L = D_f C$,

$$\boxed{\bar{\gamma} = -\bar{F}^T C_\gamma p, \quad \bar{F} = g - D_f^T \lambda}. \quad (37)$$

For cell-interpolated $\gamma_f = w_f \gamma_P + (1 - w_f) \gamma_N$, the transpose interpolation is

$$\bar{\gamma}_P = w_f \bar{\gamma}_f, \quad \bar{\gamma}_N = (1 - w_f) \bar{\gamma}_f. \quad (38)$$

Boundary values of calculated r_{AtU} vary independently in the discrete expression and are therefore reversed as explicit boundary seeds before being mapped to owner cells.

4.3 Consistent coefficient algebra

From eq. (25),

$$dr_{AtU} = \frac{r_{AtU}^2}{r_{AU}^2} dr_{AU} + r_{AtU}^2 dH_1. \quad (39)$$

Therefore the reverse update is

$$\boxed{\bar{r}_{AU} += \bar{r}_{AtU} \frac{r_{AtU}^2}{r_{AU}^2}, \quad \bar{H}_1 += \bar{r}_{AtU} r_{AtU}^2.} \quad (40)$$

For $q = r_{AtU} - r_{AU}$,

$$\bar{r}_{AtU} += \bar{q}, \quad \bar{r}_{AU} -= \bar{q}. \quad (41)$$

The q -seed receives contributions from both eq. (26) and eq. (27):

$$\bar{q}_c += \bar{H}_c \cdot (\mathcal{G}p)_c, \quad (42)$$

$$\bar{q}_f += \bar{F}_{2,f}(Np)_f, \quad (43)$$

with the face seed in the second line mapped to cells using eq. (38). The associated pressure seeds are the transposes of the cell-gradient and face-normal-gradient operators.

5 Reverse of the momentum predictor

5.1 Predictor residual identity

Differentiate eq. (20):

$$\delta Y = \delta U + \frac{r_{AU}}{V} \delta E + \frac{E}{V} \delta r_{AU}. \quad (44)$$

For an incoming seed $\bar{Y}$,

$$\bar{U} += \bar{Y}, \quad (45)$$

$$\bar{E} = \frac{r_{AU}}{V} \bar{Y}, \quad (46)$$

$$\bar{r}_{AU} = \frac{\bar{Y} \cdot E}{V}. \quad (47)$$

Because $r_{AU} = V/D_U$,

$$\delta r_{AU} = -\frac{r_{AU}^2}{V} \delta D_U, \quad (48)$$

and hence

$$\boxed{\bar{D}_U = -\bar{r}_{AU} \frac{r_{AU}^2}{V} = (\bar{Y} \cdot (U - Y)) \frac{r_{AU}}{V}.} \quad (49)$$

This is the exact seed on OpenFOAM's `UEqn.D()` scalar, including the boundary-diagonal representation used by `A()`.

5.2 Residual reverse

Write the momentum residual abstractly as

$$E = S - AU + E_{\text{explicit}}(U, \phi). \quad (50)$$

Then

$$\bar{S} += \bar{E}, \quad \bar{U} -= A^T \bar{E}, \quad (51)$$

plus the transpose of the explicit viscous and convection terms. For an internal bounded-upwind momentum face, the direct flux derivative is

$$\boxed{\bar{\phi}_f += U_f \cdot (\bar{E}_N - \bar{E}_P) + \chi (\bar{E}_P \cdot U_P - \bar{E}_N \cdot U_N).} \quad (52)$$

The explicit laminar viscous source is reversed as the composition of the transposes of divergence, deviatoric-stress, and gradient operators, with the actual patch gradient/value coefficients used at boundaries.

5.3 Diagonal and H_1 dependence on momentum flux

For an internal face and fixed upwind branch, the raw diagonal derivatives used by the bounded Gauss-upwind matrix are

$$\begin{array}{c|cc} & \partial D_P / \partial \phi_f & \partial D_N / \partial \phi_f \\ \hline \phi_f \geq 0 & 1 - \chi & \chi \\ \phi_f < 0 & -\chi & -1 + \chi \end{array} \quad (53)$$

so a seed $\bar{D}$ contributes

$$\bar{\phi}_f += \frac{1}{\alpha_U} \left(\bar{D}_P \frac{\partial D_P}{\partial \phi_f} + \bar{D}_N \frac{\partial D_N}{\partial \phi_f} \right). \quad (54)$$

For the non-coupled H_1 representation,

$$\bar{\phi}_f += \begin{cases} \bar{H}_{1,N}/V_N, & \phi_f \geq 0, \\ -\bar{H}_{1,P}/V_P, & \phi_f < 0. \end{cases} \quad (55)$$

Patch-specific coefficient rules are guarded and validated separately.

5.4 Flux interpolation transpose

From eq. (21), an incoming face seed $\bar{F}_f$ gives

$$\boxed{\bar{H}_P += w_f \bar{F}_f \mathbf{S}_f, \quad \bar{H}_N += (1 - w_f) \bar{F}_f \mathbf{S}_f.} \quad (56)$$

The reverse of `constrainHbyA` routes boundary seeds either to Y or to the old U , depending on whether the patch value is retained or overwritten.

5.5 Reverse of `adjustPhi`

For the active mass-correction branch in eq. (24), let $\bar{F}_1$ be the seed after adjustment. First

$$\bar{m} = \sum_{f \in A_+} \bar{F}_{1,f} F_{0,f}. \quad (57)$$

The incoming face seeds are

$$\bar{F}_{0,f} = \begin{cases} \bar{F}_{1,f} - \bar{m}/O_a, & f \in \mathcal{I} \cup \mathcal{F}_+, \\ m \bar{F}_{1,f} - m \bar{m}/O_a, & f \in \mathcal{A}_+. \end{cases} \quad (58)$$

When pressure does not need a reference, $F_1 = F_0$ and the reverse is the identity.

6 One-step transpose and fixed-point adjoint

6.1 Exact one-step reverse

The reverse operations in eqs. (34), (35), (37), (40), (41), (46), (49), (52), (54) to (56) and (58) are composed in reverse order to obtain

$$\boxed{\bar{x} = M_x^T \bar{x}^+} \quad (59)$$

The defining validation identity is

$$\langle y, M_x d \rangle = \langle M_x^T y, d \rangle \quad (60)$$

for independent state directions spanning U, p, ϕ_i, ϕ_b .

6.2 Fixed-point tangent equation

At convergence,

$$x = M(x, \beta). \quad (61)$$

Differentiating with respect to the design gives

$$(I - M_x)x_\beta = M_\beta. \quad (62)$$

Directly solving eq. (62) for every design variable would be prohibitively expensive.

6.3 Fixed-point adjoint equation

The reduced objective derivative is

$$\frac{dJ}{d\beta} = J_\beta + J_x x_\beta. \quad (63)$$

Define the fixed-point adjoint by the standard reverse-accumulation construction for a converged iteration [1]:

$$\boxed{(I - M_x)^T \psi = J_x.} \quad (64)$$

This requires $I - M_x$ to be nonsingular. A sufficient local condition is $\rho(M_x) < 1$, in which case the Neumann series exists. Primal convergence is useful empirical evidence of local stability but is not, by itself, a formal proof of contractivity. In the validated case, well-posedness is supported numerically by repeatable GMRES convergence and true relative residuals below 10^{-9} .

Multiplying eq. (62) by ψ^T and using eq. (64) gives

$$\boxed{\frac{dJ}{d\beta} = J_\beta + \psi^T M_\beta.} \quad (65)$$

The matrix-free operator applied by GMRES is

$$\mathcal{K}(v) = v - M_x^T v. \quad (66)$$

A weighted inner product is used for numerical conditioning,

$$\begin{aligned} \langle a, b \rangle_W &= s_U^2 a_U^T b_U + s_p^2 a_p^T b_p \\ &\quad + s_\phi^2 (a_{\phi_i}^T b_{\phi_i} + a_{\phi_b}^T b_{\phi_b}). \end{aligned} \quad (67)$$

Writing $W = \text{diag}(s_U^2, s_p^2, s_\phi^2, s_\phi^2)$, this is equivalent to Euclidean GMRES in the diagonally scaled coordinates $W^{1/2}x$. It changes the Krylov geometry and residual norm, but not the exact converged solution of eq. (64). All physical transpose identities use the unweighted algebraic pairing eq. (8).

6.4 Solved-adjoint identity

For any state direction d , eq. (64) implies

$$\boxed{\langle J_x, d \rangle = \langle \psi, (I - M_x)d \rangle.} \quad (68)$$

This identity simultaneously checks the objective seed, the one-step transpose, pressure gauge projection, and the fixed-point linear solve.

7 Analytic design reverse $M_\beta^T \psi$

7.1 Where beta enters the one-step map

In the validated branch, beta enters the momentum matrix through a local implicit source coefficient $a(\beta)$, assembled as an `fvm::Sp` contribution. At fixed state,

$$\frac{\partial D_{\text{raw}}}{\partial a} = V, \quad (69)$$

and there is no beta dependence in the momentum off-diagonal coefficients or H_1 .

For cell c , define the relaxation off-diagonal measure used by the validated scalar-diagonal branch as

$$\Sigma_{\text{off},c} = \sum_{j \neq c} |A_{cj}|, \quad (70)$$

including the applicable boundary-coefficient contribution in the same component convention used by `fvmatrix::relax`. OpenFOAM equation relaxation selects

$$\hat{D}_c = \max(|D_{\text{raw},c}|, \Sigma_{\text{off},c}), \quad (71)$$

then divides by the velocity relaxation factor α_U and adds the diagonal change times the old velocity to the source. On the strict positive raw-diagonal branch,

$$\frac{\partial D_{\text{final}}}{\partial a} = \frac{V}{\alpha_U}, \quad (72)$$

$$\frac{\partial S_{\text{final}}}{\partial a} = \left(\frac{V}{\alpha_U} - V \right) U, \quad (73)$$

$$\frac{\partial H_1}{\partial a} = 0. \quad (74)$$

7.2 Feasible derivative at structural lower-bound ties

At a structural tie,

$$\beta = 0, \quad D_{\text{raw}} = \Sigma_{\text{off}} > 0. \quad (75)$$

Only inward feasible perturbations $\delta\beta \geq 0$ are admissible. The audited source interpolation satisfies $a'_+(0) > 0$, while Σ_{off} is beta independent. Therefore

$$\delta D_{\text{raw}} > 0, \quad \delta \Sigma_{\text{off}} = 0, \quad (76)$$

and the feasible right derivative of eq. (71) selects the raw-diagonal branch. Consequently eqs. (72) to (74) remain valid as feasible right derivatives at these cells.

7.3 Reverse trace and coefficient contraction

The exact one-step reverse exposes the following seeds:

$$\bar{S}_{\text{final}} = \bar{E}, \quad (77)$$

$$\bar{D}_{\text{res}} = -(\bar{E} \cdot U), \quad (78)$$

$$\bar{D}_r = \text{all downstream seeds on final } D \text{ through } r_{AU}, r_{AtU}, q, \text{ and pressure}, \quad (79)$$

$$\bar{H}_1 = \text{the downstream consistent-SIMPLE } H_1 \text{ seed}. \quad (80)$$

Let

$$\bar{D}_{\text{tot}} = \bar{D}_{\text{res}} + \bar{D}_r. \quad (81)$$

The derivative with respect to the local raw source coefficient is

$$\boxed{g_a = \bar{D}_{\text{tot}} \frac{V}{\alpha_U} + \bar{S}_{\text{final}} \cdot \left(\frac{V}{\alpha_U} - V \right) U + \bar{H}_1 0.} \quad (82)$$

Using eqs. (77) and (78), the residual-diagonal and relaxation-source terms simplify exactly:

$$\bar{D}_{\text{res}} \frac{V}{\alpha_U} + \bar{E} \cdot \left(\frac{V}{\alpha_U} - V \right) U = \bar{D}_{\text{res}} V. \quad (83)$$

Hence the independently checked simplified form is

$$\boxed{g_a = \bar{D}_{\text{res}} V + \bar{D}_r \frac{V}{\alpha_U}.} \quad (84)$$

The implementation retains the full expression eq. (82) and uses eq. (84) only as a diagnostic identity.

7.4 Mapping to physical beta and optimiser variables

The local coefficient sensitivity is passed through the same source interpolation and topology post-processing as the primal source:

$$g_{\beta}^{\text{flow,int}} = \left(\frac{\partial a}{\partial \beta} \right)^T g_a, \quad (85)$$

with filter/projection transposes included when the optimiser variable differs from physical beta. The OpenFOAM `betaMult` convention is

$$\boxed{g_{\beta,c}^{\text{flow,betaMult}} = \frac{\Delta t}{V_c} g_{\beta,c}^{\text{flow,int}}.} \quad (86)$$

8 Direct conductivity derivative J_{β}

8.1 Internal-face discrete derivative

For an internal thermal diffusion face in the validated orthogonal/two-point branch, let

$$\gamma_f = |S_f| \delta_f, \quad D_f = w_f D_P + (1 - w_f) D_N, \quad (87)$$

where δ_f is the face-normal delta coefficient used by the orthogonal finite-volume Laplacian. The assembled residual contributions are

$$R_P -= D_f \gamma_f (T_N - T_P), \quad (88)$$

$$R_N += D_f \gamma_f (T_N - T_P). \quad (89)$$

Using $\mathcal{L}_T = J + T_a^T R_T$,

$$\boxed{\frac{\partial \mathcal{L}_T}{\partial D_f} = \gamma_f (T_N - T_P) (T_{a,N} - T_{a,P}).} \quad (90)$$

The face derivative is distributed to cells by the same interpolation weights:

$$g_{D,P} += w_f g_{D,f}, \quad g_{D,N} += (1 - w_f) g_{D,f}. \quad (91)$$

If a corrected non-orthogonal Laplacian is active, the residual also contains an explicit correction flux proportional to D_f . Its derivative contributes an additional face term and, because the correction can depend on reconstructed gradients, its complete transpose must be included. Equation eq. (90) is therefore an exact complete conductivity derivative only when the explicit non-orthogonal correction is zero (orthogonal mesh or uncorrected scheme), as required by the validated branch.

8.2 Boundary-face derivative

For a non-coupled boundary face with extrapolated conductivity indicator,

$$\boxed{g_{D,P} -= T_{a,P} |S_f| \text{snGrad}(T)_f.} \quad (92)$$

This term vanishes on adiabatic boundaries and is nonzero on a heated wall, where beta changes the boundary heat input through the diffusivity.

8.3 Chain through the conductivity interpolation

From eq. (4),

$$\frac{\partial D_{\text{eff}}}{\partial I_k} = D_s - D_f - \frac{\nu_t}{\text{Pr}_t}. \quad (93)$$

Therefore the per-volume, time-scaled physical sensitivity before the topology chain is

$$\boxed{g_{\beta,c}^{\text{cond}} = \frac{\Delta t}{V_c} \text{thermalSensScale } g_{D,c} \left(D_s - D_f - \frac{\nu_t}{\text{Pr}_t} \right) I'_k(\beta_c).} \quad (94)$$

The implementation applies the active-design mask before and after the topology interpolation transpose so fixed zones and inlet/outlet-adjacent cells cannot acquire a nonzero optimiser sensitivity.

9 Production assembly and avoidance of double counting

The exact total design multiplier is

$$\boxed{\text{betaMult}_{\text{final}} = \text{betaMult}_{\text{before}} + g_{\beta}^{\text{flow,betaMult}} + g_{\beta}^{\text{cond}}.} \quad (95)$$

The standard OpenFOAM adjoint first produces a stock Brinkman momentum contribution. In exact fixed-point mode, that stock contribution is *replaced*, not retained:

$$\text{betaMult} \leftarrow \text{betaMult}_{\text{before}}, \quad (96)$$

$$\text{betaMult} \leftarrow \text{betaMult} + g_{\beta}^{\text{flow,betaMult}}, \quad (97)$$

$$\text{betaMult} \leftarrow \text{betaMult} + g_{\beta}^{\text{cond}}. \quad (98)$$

Adding the analytic field on top of the stock contribution would double count the momentum design effect and is explicitly guarded against.

10 Validation identities and numerical evidence

10.1 Tangent residual-correction identity

Suppose an approximate tangent δx has residual

$$r = M_\beta - (I - M_x)\delta x. \quad (99)$$

Using eq. (64),

$$\boxed{J_x^T \delta x + \psi^T r = \psi^T M_\beta.} \quad (100)$$

This identity proved that earlier tangent/adjoint discrepancies were caused by the tangent solver's nonzero true residual rather than an error in the analytic adjoint contraction.

10.2 All-cell design-direction identity

For any feasible design direction $d\beta$,

$$\boxed{\psi^T (M_\beta d\beta) = (M_\beta^T \psi)^T d\beta.} \quad (101)$$

The left side is evaluated by one-sided or central finite differences of the complete one-step map, including all four state blocks. The right side is the dot product of the analytic all-cell field with the same design direction.

10.3 Bound-cell formulas

At $\beta = 0$, a second-order feasible forward derivative is

$$M_\beta d \approx \frac{-3M(\beta) + 4M(\beta + hd) - M(\beta + 2hd)}{2h}. \quad (102)$$

The fourth-order corroborating formula is

$$M_\beta d \approx \frac{-25M_0 + 48M_1 - 36M_2 + 16M_3 - 3M_4}{12h}. \quad (103)$$

For the second-order estimates $D_2(h)$, Richardson extrapolation is

$$D_R(h, h/2) = \frac{4D_2(h/2) - D_2(h)}{3}. \quad (104)$$

These formulas were essential for resolving the pressure-solve finite-difference floor at a lower-bound test cell.

10.4 Production qualification summary

Table 1: Final production qualification results supplied by the validated run.

Check	Result
One-step tape reproduction	0
Base fixed-point defect	4.30×10^{-10}
Fixed-point thermal adjoint relative residual	9.86×10^{-10}
Random RHS relative residuals	9.25×10^{-10} ; 9.51×10^{-10}
Operator state preservation / determinism	0 / 0
Operator linearity relative differences	approximately 5.3×10^{-13} to 7.1×10^{-13}
Trace no-change / production replacement / total assembly	0 / 0 / 0
Structural relaxation classes	820 strict raw; 380 feasible lower-bound ties; 0 unsupported
Three-cycle fixed-point iterations	218; 226; 210
Three-cycle objective trend	$3.226 \times 10^{-4} \rightarrow 3.122 \times 10^{-4}$ $10^{-4} \rightarrow 3.112 \times 10^{-4}$
Fixed-zero and IO-cell maximum design change	0
Baseline disabled-mode comparison	byte-identical sensitivity and primal fields
Validation-on versus production-only	byte-identical sensitivity and primal fields

11 Implementation map

The equations above map to the production implementation by function name. Line numbers are intentionally omitted because they are not stable under editing. **Validated source snapshot:** commit 1b7778f9abd747aea478a71acfd5a44c093ba77, tag v0.4.3.

Runtime enforcement of the validated scope. The production implementation does not rely on these assumptions being remembered by the user. When `useAnalyticFixedPointAdjointBeta` is enabled, runtime guards require constant fluid and solid diffusivities and constant volumetric heat capacity; exactly one active `top0Source` momentum option and no unsupported momentum or temperature `fvOptions`; zero pressure non-orthogonal correctors; serial execution; Stokes flow; consistent SIMPLE; and non-coupled patches. A corrected thermal `snGrad` scheme is accepted only when its measured explicit non-orthogonal correction is roundoff-zero. These checks are implemented by `validateProductionFixedPointBetaScope()` and `validateThermalDiffusionCorrectionScope()`, with property-mode introspection supplied by `thermalPropertyTables`.

1. **Assembled thermal transpose, eq. (13):** `solveTaEqn()` in the exact scalar branch.
2. **Thermal flux seed, eqs. (16) and (17):** `thermalFluxSensitivity()`.
3. **One-step map M , eq. (32):** `primalSimpleMapStateAtFrozenState()`.
4. **One-step transpose M_x^T , eq. (59):** `reverseOneSimpleMapSeedImpl()`.
5. **Fixed-point RHS, eq. (18):** `fixedPointObjectiveSeed()`.
6. **Operator $I - M_x^T$, eq. (66):** `applyFixedPointAdjointOperator()`.

7. **GMRES solve, eq. (64):** `solveFixedPointAdjointSystem()`.
8. **Analytic $M_\beta^T \psi$, eqs. (82) and (86):** `analyticFixedPointAdjointBeta()`.
9. **Direct conductivity, eqs. (90) and (94):** the face-based conductivity block in `topOSensMultiplier()`.
10. **Production replacement and assembly, eq. (95):** `topOSensMultiplier()`.

12 What is exact, and what remains branch-dependent

- The flow adjoint is the transpose of one actual SIMPLE update, including pressure relaxation, consistent-SIMPLE coefficient algebra, flux and boundary states, `adjustPhi`, `constrainPressure`, `constrainHbyA`, and the exact OpenFOAM diagonal representation.
- The fixed-point adjoint accounts for repeated SIMPLE iterations through $(I - M_x)^T \psi = J_x$; it is not the same object as the stock residual-adjoint velocity field.
- The thermal equation is outside the flow fixed-point state because coupling is one-way. Buoyancy, temperature-dependent momentum properties, or any other $T \rightarrow (U, p, \phi)$ path require an enlarged fixed-point state.
- The implemented scalar transpose is exact for the assembled constant-property or frozen-property Picard operator. It is not the full nonlinear thermal-property adjoint unless the derivative terms in eq. (11) vanish or are explicitly included.
- The direct conductivity formula is exact for the validated orthogonal/uncorrected diffusion branch. A nonzero explicit non-orthogonal correction flux requires its own reverse term. A non-orthogonal pressure-corrector loop likewise changes the one-step map and is outside the current production scope.
- The design derivative at strict raw-diagonal cells is classical. At structural lower-bound relaxation ties it is the unique feasible right derivative for increasing the Brinkman/source coefficient.
- Upwind, `adjustPhi`, and other discrete switching points are piecewise differentiable. A derivative is valid only within the active discrete branch or as an explicitly defined feasible directional derivative. Adjustable negative boundary fluxes are inflow; exactly zero fluxes are branch points.
- The current production claim does not extend to parallel execution, non-empty coupled patches, turbulent models other than the validated `Stokes` branch, arbitrary scalar convection schemes, or objectives with direct flow-state derivatives.

13 Final formula

Define

$$M_a = \frac{\partial M}{\partial a}, \quad J_{I_k} = \frac{\partial \mathcal{L}_T}{\partial I_k}, \quad (105)$$

where J_{I_k} includes the face-to-cell conductivity contraction before multiplication by $I'_k(\beta)$. Combining the exact flow fixed-point contribution and the direct thermal-conductivity contribution gives

$$\boxed{\frac{dJ}{d\rho} = \left(\frac{\partial \beta}{\partial \rho}\right)^T \left[\left(\frac{\partial a}{\partial \beta}\right)^T M_a^T \psi + \left(\frac{\partial I_k}{\partial \beta}\right)^T J_{I_k} \right], \quad (I - M_x)^T \psi = J_x.} \quad (106)$$

In OpenFOAM's production convention, the two bracketed fields are divided by cell volume, multiplied by the topology time-step factor, masked to the active design space, and assembled according to eq. (95).

Interpretation. For the explicitly guarded constant-property, one-way-coupled, orthogonal/un-corrected branch, the final sensitivity is the derivative of the objective produced by the converged discrete solver with respect to the topology design. It includes the effect of the design on every future SIMPLE iteration through the fixed-point adjoint and the direct face-based conductivity derivative. With temperature-dependent thermal properties, the current scalar transpose gives the derivative of the frozen-property Picard problem unless the additional terms in eq. (11) are implemented.

References

- [1] B. Christianson, “Reverse accumulation and attractive fixed points,” *Optimization Methods and Software*, vol. 3, no. 4, pp. 311–326, 1994. [doi:10.1080/10556789408805572](https://doi.org/10.1080/10556789408805572).
- [2] E. M. Papoutsis-Kiachagias and K. C. Giannakoglou, “Continuous adjoint methods for turbulent flows, applied to shape and topology optimization: industrial applications,” *Archives of Computational Methods in Engineering*, vol. 23, no. 2, pp. 255–299, 2016. [doi:10.1007/s11831-014-9141-9](https://doi.org/10.1007/s11831-014-9141-9).
- [3] T. Borrvall and J. Petersson, “Topology optimization of fluids in Stokes flow,” *International Journal for Numerical Methods in Fluids*, vol. 41, no. 1, pp. 77–107, 2003. [doi:10.1002/fld.426](https://doi.org/10.1002/fld.426).

A Compact derivation of the central fixed-point result

For reference, the entire fixed-point derivation can be compressed to four lines:

$$x - M(x, \beta) = 0, \tag{107}$$

$$(I - M_x)x_\beta = M_\beta, \tag{108}$$

$$(I - M_x)^T \psi = J_x, \tag{109}$$

$$\frac{dJ}{d\beta} = J_\beta + J_x x_\beta = J_\beta + \psi^T (I - M_x)x_\beta = J_\beta + \psi^T M_\beta. \tag{110}$$

All remaining work in this report is the exact construction and validation of M_x^T , J_x , $M_\beta^T \psi$, and J_β for the actual OpenFOAM finite-volume operations.